

1. Output is based on character so it falls under classification
2. Total rows-399, Total columns-25, 1-output, 24-input
3. Among all models SVM is the best parameter **C:10, gamma:auto, kernel:Sigmoid**

F1 score-**0.99**

Accuracy-**0.99**

ROC Score-**1.0**

SVM

```
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, grid_predictions)
```

```
from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_mac
```

```
.. \ best parameter {'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}: 0.9924946382275899
```

```
print("The confusion Matrix:\n",cm)
```

```
... The confusion Matrix:
[[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	51
1	1.00	0.99	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

```
print('ROC AUC Score:', roc_auc_score(y_test,grid.best_estimator_.predict_proba(X_
```

```
ROC AUC Score: 1.0
```

Decision Tree

```
from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)
```

The f1_macro value for best parameter {'criterion': 'entropy', 'max_features': 'log2', 'splitter': 'random'}: 0.9548872180451128

```
print("The confusion Matrix:\n",cm)
```

The confusion Matrix:
[[48 3]
 [3 79]]

```
print("The report:\n",clf_report)
```

The report:

	precision	recall	f1-score	support
0	0.94	0.94	0.94	51
1	0.96	0.96	0.96	82
accuracy			0.95	133
macro avg	0.95	0.95	0.95	133
weighted avg	0.95	0.95	0.95	133

```
from sklearn.metrics import roc_auc_score
```

```
roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])
```

```
np.float64(0.9522955523672884)
```

Random Forest

```
arn.metrics import f1_score
f1_score(y_test,grid_predictions,average='weighted')
e f1_macro value for best parameter {}".format(grid.best_params_),f1_macro)

... 'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 100}: 0.984962406015037
```

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
[[50  1]
 [ 1 81]]
```

Logistic Regression

```
from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,grid_predictions,average='weighted')
print("The f1_macro value for best parameter {}".format(grid.best_params_),f1_ma

... alue for best parameter {'penalty': 'l2', 'solver': 'newton-cg'}: 0.99249463822758
```

```
print("The confusion Matrix:\n",cm)
```

```
The confusion Matrix:
[[51  0]
 [ 1 81]]
```

```
print("The report:\n",clf_report)
```

```
The report:
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	51
1	1.00	0.99	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test,grid.predict_proba(X_test)[:,:1])

np.float64(1.0)
```

KNN

```
print(clf_report)
```

	precision	recall	f1-score	support
0	0.86	1.00	0.93	51
1	1.00	0.90	0.95	82
accuracy			0.94	133
macro avg	0.93	0.95	0.94	133
weighted avg	0.95	0.94	0.94	133

Guassian NB

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
print(cm)
```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	51
1	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

```
[[51  0]
 [ 3 79]]
```

```

from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
print(cm)

```

	precision	recall	f1-score	support
0	0.94	1.00	0.97	51
1	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

```

[[51  0]
 [ 3 79]]

```